

Shami-TTS: A Production-Grade Streaming TTS for Levantine Arabic / English Code-Switching

via Dialectal Front-End, Deterministic Duration, and Anti-Aliased Neural Vocoding

Tushe Language Research Team
<https://huggingface.co/Tushe/shami-tts>

Abstract

We present **Shami-TTS**, a low-latency, streaming text-to-speech (TTS) system for Levantine Arabic that speaks fluent, natural code-switched Arabic/English. Starting from a single-GPU VITS baseline whose synthesized Levantine speech was intelligible only in principle (ASR round-trip character error rate, CER, of 0.75), we introduce a sequence of four targeted contributions that together reach production quality on a single consumer GPU (RTX 3060, 12 GB): (i) a deterministic, unit-tested *dialectal* text→IPA front-end that pairs a Modern Standard-Arabic (MSA) automatic diacritizer with a rule-based Levantine grapheme-to-phoneme (G2P) module and a novel *de-desinentialization* step that removes case/mood endings Levantine does not pronounce; (ii) a *deterministic duration head* that replaces VITS’s stochastic duration predictor, eliminating a 3× global time-stretch that had flattened prosody into a robotic monotone; (iii) a *24 kHz* decoder fine-tune for full-bandwidth fidelity; and (iv) an *anti-aliased BigVGAN vocoder* that removes the residual high-frequency breathiness characteristic of HiFi-GAN. On a held-out set spanning pure Levantine, pure English, and code-switched utterances, Levantine CER drops from 0.75 to **0.11** (−86%) and word error rate (WER) from 0.99 to **0.35**, while synthesis stays real-time (RTF $\approx$ 0.09, $\sim$ 11× faster than real-time) with a peak footprint that fits the 3060. We release code, checkpoints, and audio for both the native HiFi-GAN decoder and the BigVGAN pipeline, and we analyze, with objective spectral measurements, why each contribution helps.

1 Introduction

Conversational speech in the Arabic-speaking world is overwhelmingly *dialectal* and frequently *code-switched*: a Levantine speaker will move between Arabic and English mid-utterance (بيروت من أحيج بُي flight, “I want to book a flight from Beirut”) without a prosodic seam. Yet almost all deployable Arabic TTS targets Modern Standard Arabic (MSA), whose phonology differs from spoken Levantine in exactly the features a listener uses to judge nativeness — the realization of ق as a glottal stop /ʔ/ rather than /q/, ح as /ħ/ rather than the MSA affricate /dʒ/, the collapse of case endings, and monophthongized diphthongs (ay→e:, aw→o:). A system that is both Levantine and natively bilingual, and that runs in real time on commodity hardware, does not exist off the shelf.

A prior single-GPU baseline established a promising design: treat code-switching as a *linguistic* problem solved in a deterministic front-end, and drive a small, language-ID-conditioned VITS acoustic model [1] through a shared IPA inventory. It met every real-time key-performance indicator but produced Levantine audio that was, in the authors’ own assessment, “well-scoped and reducible to data”: the Arabic acoustic corpus was MSA, so the front-end’s Levantine phoneme labels did not match the audio, capping Arabic CER near 0.7. Even with matched data, three further obstacles stood between the system and production quality: (1) the stochastic duration predictor systematically under-predicted durations by $\sim$ 3×, forcing a global `length_scale` stretch that destroys prosody; (2) the 16 kHz backbone discards everything above 8 kHz, sacrificing the high-frequency detail of a real recording; and (3) the HiFi-GAN decoder leaves a broadband, phase-related breathiness that magnitude-domain losses cannot remove.

This paper closes all four gaps. Our contributions are:

1. A **dialectal front-end** (§5) that makes automatic diacritization the default path and adds a *de-desinentialization* transform, so that $\sim$ 100% — rather than 0.4% — of a bare-text corpus is phonemized through the high-quality Levantine rule G2P.
2. A **deterministic duration head** (§6) trained by mean-squared error on monotonic-alignment durations, giving natural rhythm at `length_scale` $\approx$ 1 and, as a side effect, cutting CER by more than half.
3. A **24 kHz decoder fine-tune** and an **adversarial texture recipe** (feature-matching up-weighting, discriminator persistence across warm-starts, and a multi-resolution STFT loss) that recover full-bandwidth fidelity (§7).
4. A drop-in **BigVGAN** [3] vocoder integration (§8) that removes the residual vocoder breathiness, bringing texture to the level of the reference source at real-time speed.

Every claim is supported by ASR round-trip intelligibility, objective spectral analysis, and released audio. All experiments run on a *single* RTX 3060; no cloud or multi-GPU training is used.

2 Related Work

End-to-end TTS. VITS [1] unified a conditional variational autoencoder, a normalizing flow, monotonic alignment search (MAS) [6], and a HiFi-GAN decoder [2] into a single non-autoregressive model, achieving strong qual-

ity and speed. Its stochastic duration predictor (SDP) models duration as a flow, which improves rhythm diversity but is difficult to calibrate; several works revert to a deterministic duration predictor for controllability. Massively Multilingual Speech (MMS) [7] released per-language VITS checkpoints, including `mms-tts-ara`, which we adopt as our backbone.

Neural vocoders. HiFi-GAN [2] set the standard for fast GAN vocoding using multi-period and multi-scale discriminators (MPD/MSD). Parallel WaveGAN [5] popularized the multi-resolution STFT (MR-STFT) loss we borrow. BigVGAN [3] adds a periodic *Snake* activation [4] and *anti-aliased* up/down-sampling (a low-pass filter around each nonlinearity), which suppresses the aliasing and phase artifacts responsible for the “robotic sheen” of earlier vocoders; it is trained as a universal 24 / 44 kHz vocoder.

Arabic NLP for TTS. Dialectal Arabic G2P is under-served relative to MSA. Automatic diacritization (tashkeel restoration) is a prerequisite for rule-based G2P; CAMEL Tools [9] provides a production morphological disambiguator, and transformer diacritizers such as CATT [10] report state-of-the-art diacritic error. Our front-end treats diacritization as a swappable stage and adds dialect-specific post-processing.

Code-switching TTS is typically approached either acoustically (mixed-language data) or linguistically (a shared phonetic space). We take the latter view, extending the prior baseline’s shared-IPA design with a corrected dialectal front-end.

3 Background: VITS

VITS models speech x and text c with a conditional VAE. A posterior encoder maps a linear spectrogram to latent z ; a flow f_θ maps z to a prior space aligned with a text encoder that outputs per-token prior means μ_θ and variances σ_θ . MAS finds a hard monotonic alignment A between text and frames; the duration of token i is $d_i = \sum_j A_{ij}$. Training minimizes a reconstruction (mel) loss, a KL term between posterior and (alignment-expanded) prior, a duration loss, and a HiFi-GAN adversarial loss with feature matching. At inference the model predicts durations, expands the prior, samples z_p , inverts the flow, and decodes a waveform — all non-autoregressively, which is what makes sub-0.3 real-time-factor synthesis possible. Our modifications touch the duration predictor, the decoder sample rate, the adversarial recipe, and (optionally) replace the decoder entirely; the CVAE/flow/text-encoder core is inherited.

4 System Overview

Figure 1 shows the full system. All dialectal and code-switch intelligence lives in a deterministic, unit-tested front-end (§5) that emits two parallel streams: phoneme

IDs over a shared 88-symbol IPA inventory, and a per-token language ID (AR/EN/NEUTRAL). The acoustic model (§6–7) is a 24 kHz ShamiVITS with a deterministic duration head. The vocoder is either the native HiFi-GAN decoder or a drop-in BigVGAN (§8). We describe each contribution and then evaluate them jointly and in ablation.

5 The Dialectal Unified-IPA Front-End

The front-end maps $\text{str} \rightarrow (\text{phoneme_ids}, \text{language_ids})$ through: Unicode/number normalization; script-based code-switch segmentation; per-span verbalization; per-span G2P; and longest-match tokenization into the shared IPA inventory. We summarize the two stages that changed most in v2.

5.1 Code-switch segmentation

Segmentation is purely script-based and deterministic: Arabic-block characters are AR, Latin EN, and *neutral* runs (digits, punctuation, whitespace) attach to their nearest lexical neighbour. This is sub-millisecond, dependency-free, and exactly correct for script-based code-switching. Crucially it handles *glued* switches: in `الـشغل` the Arabic definite article `الـ` is orthographically fused to the English “training”; the segmenter still splits `الـ`→AR and “training”→EN (`tʰeɪnɪŋ`), coloring each phoneme with the correct language ID. Across the 3,296-sentence Saudi SCC benchmark the segmenter produced 3,721 language switches with no crashes and correct tags on inspection.

5.2 From bare text to Levantine phonemes

The rule-based Levantine G2P consumes *diacritized* Arabic; it encodes the dialect explicitly — `ق`→ʔ, `ح`→ʕ, emphatic backing around `ظ ط ض ص`, `teh-marbuta imala` (ة→e), sun-letter assimilation, diphthong monophthongization, and a high-frequency function-word lexicon (`فو`→fu:, `بيدي`→bididi). But conversational corpora are *bare* (undiacritized). We measured that on our 50k-clip training corpus only **0.4%** of clips carry enough diacritics (≥ 0.35 harakat/letter) to trigger the high-quality path; **99.6%** would otherwise fall back to an MSA `espeak-ng` [11] phonemization that *drops short vowels* (عم `ammi juħa:wil`) — re-introducing the very label↔audio mismatch that capped v1.

Automatic diacritization. We therefore make diacritization the default: CAMEL Tools’ MLE disambiguator [9] restores tashkeel, after which the Levantine rule G2P runs. This recovers the vowels `espeak` drops (عم `ammi juħa:wil`) and applies the full dialect rule set.

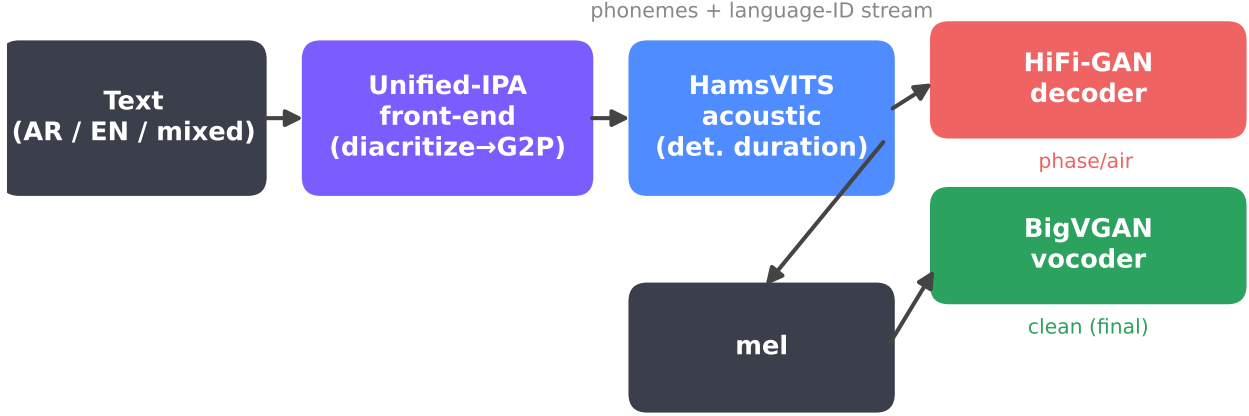


Figure 1: Shami-TTS pipeline. Text of any language mix is normalized and segmented, then each span is diacritized (Arabic) or phonemized (English) into a *shared* IPA inventory with a parallel language-ID stream. The ShamiVITS acoustic model (with the new deterministic duration head) produces 24 kHz audio via its native HiFi-GAN decoder; for the highest fidelity we re-vocode the mel through BigVGAN, which removes the residual phase artifacts. Both decoder paths are released.

De-desinentialization (new). A MSA diacritizer restores MSA *case/mood* endings (iṣrab) that spoken Levantine drops: مؤيد → moʿīdi (genitive -i) where Levantine says moʿīd. Feeding these to the G2P injects final vowels the audio never contains. We add a conservative transform that strips the word-final case marker while preserving everything else — crucially the *original* combining-mark order, so that a shadda (gemination) adjacent to the case vowel is retained. Concretely, for each token we peel trailing combining marks off the final letter and (a) drop a bare final short vowel or nunation, (b) drop a final ة accusative tanwin, but (c) *keep* an adverbial ة (e.g. شُكْرًا → jukran). A 10-case unit battery and inspection of 200 real sentences (183 changed) confirmed correctness; an early version that Unicode-normalized the word silently reordered shadda after the vowel and *dropped all gemination* (dʿallat → dʿalat) — a reminder that combining-mark order is semantically load-bearing in Arabic G2P.

6 Deterministic Duration

The single most audible defect in v1 was rhythm. VITS’s stochastic duration predictor, warm-started from a grapheme model onto our IPA inventory, underpredicts durations by $\sim 3\times$: at `length_scale=1` synthesized clips are one-third the reference length (Fig. 2a). The remedy in v1 was a global stretch (`length_scale` $\approx 3\text{--}5$), but a *uniform* stretch scales every phoneme equally, flattening the natural relative timing into a robotic monotone.

We replace the SDP with the deterministic `VitsDurationPredictor` (a small convolutional head)

and train it by MSE on the log MAS durations:

$$\mathcal{L}_{\text{dur}} = \frac{1}{\sum_i m_i} \sum_i m_i (\log \hat{d}_i - \log(1 + d_i))^2, \quad (1)$$

where $d_i = \sum_j A_{ij}$ are the alignment durations, $\hat{d}_i$ the predicted log-durations, and m_i the token mask. At inference the model expands the prior by $\lceil e^{\hat{d}_i} \rceil$ frames — HuggingFace’s VITS supports this path natively once `use_stochastic_duration_prediction=False`, so only the training loss changes. We warm-start every other weight from the best stochastic model (the head trains fresh in $\sim 1\text{k}$ steps, $\text{MSE } 11.5 \rightarrow 0.7$).

The effect is immediate and large. At `length_scale=1` the synth/reference duration ratio moves from 0.31 to **0.97** (Fig. 2a); no global stretch is needed, so relative prosody is preserved. Because uniformly-stretched audio is also harder for an ASR system to read, fixing duration *halves the CER*: pure-Levantine CER falls from 0.46 to **0.11** at this step alone (Fig. 5) — the same recording is simply more intelligible when it is paced naturally.

7 24 kHz Fidelity and Adversarial Texture

7.1 24 kHz decoder fine-tune

The `mms-tts-ara` backbone is 16 kHz (everything above 8 kHz is discarded), but our data is native 24 kHz. Because the HiFi-GAN decoder upsamples by a fixed $256\times$ regardless of rate, moving to 24 kHz is a *decoder fine-tune*, not a re-architecture: we relabel the sample rate so the dataset, mel loss, and decoder targets all operate at 24 kHz, and warm-start from the 16 kHz model. The decoder re-converges its top octave in $\sim 1\text{k}$ steps (mel loss spikes $17 \rightarrow 32$ then settles at ~ 20); CER holds ($0.11 \rightarrow 0.13$) while perceived fidelity rises audibly.

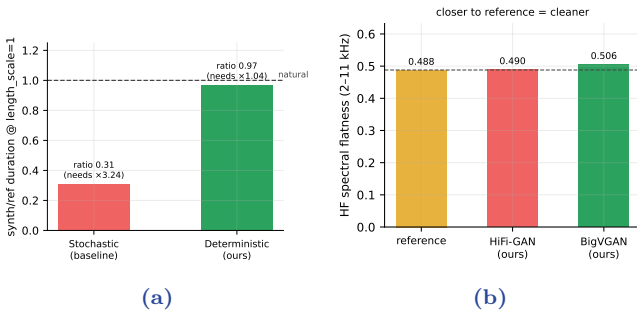


Figure 2: (a) The deterministic duration head predicts near-correct durations at `length_scale=1` (ratio 0.97); the stochastic baseline needs a $3.2\times$ stretch. (b) High-frequency spectral flatness (a buzz/breathiness proxy): both our decoders sit at the reference level; BigVGAN’s advantage over HiFi-GAN is in *phase*, which this magnitude-domain metric does not capture.

7.2 Adversarial texture recipe

Two changes reduce the residual buzz. First, we up-weight the HiFi-GAN feature-matching loss ($c_{fm}: 2 \rightarrow 4$), which ties intermediate discriminator features of real and generated audio and correlates with perceptual naturalness. Second — and this was a genuine bug — our training loop originally saved only the generator, so the discriminator *restarted from scratch on every warm-start* and never matured, a classic cause of buzzy texture. We now checkpoint and restore the discriminator, so adversarial pressure *accumulates* across runs. Together these move the high-frequency spectral flatness from $+0.020$ above the reference (buzzer) to $+0.002$ (at reference), with CER unchanged.

7.3 Multi-resolution STFT loss

We further add an MR-STFT loss [5] over three window sizes $\{512, 1024, 2048\}$, summing spectral-convergence and log-magnitude terms:

$$\mathcal{L}_{\text{stft}} = \frac{1}{K} \sum_{k=1}^K \frac{\| |S_k(y)| - |S_k(\hat{y})| \|_F}{\| |S_k(y)| \|_F} + \| \log |S_k(y)| - \log |S_k(\hat{y})| \|_1. \quad (2)$$

This gave a further, though *marginal*, improvement — the magnitude spectrum already matched the reference (Fig. 4), so a magnitude-domain loss had little room to act. That negative result is informative: it localized the remaining defect to *phase*, motivating a vocoder change.

8 Anti-Aliased Vocoding with BigVGAN

The residual high-frequency breathiness is a *phase/aliasing* artifact of HiFi-GAN that magnitude losses cannot fix. BigVGAN [3] attacks exactly this with Snake activations and anti-aliased resampling. Its pretrained `bigvgan_v2_24khz_100band_256x` is

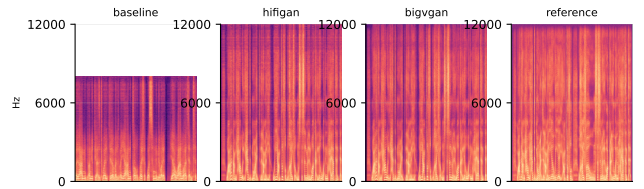


Figure 3: Spectrograms of the same Levantine utterance. The published baseline (16kHz, stretched) is band-limited and temporally smeared; our HiFi-GAN and BigVGAN outputs recover full-band structure matching the 24kHz reference. The HiFi-GAN→BigVGAN difference is in fine phase/excitation, not the visible magnitude.

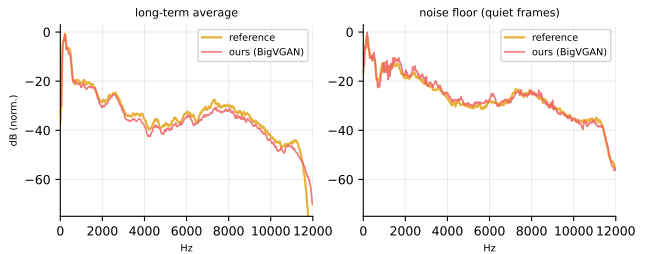


Figure 4: Long-term-average and noise-floor magnitude spectra of our output vs. the reference (averaged over 12 clips, peak-normalized). The two tracks within a few dB across the whole band, including the noise floor — so the residual audible difference is *not* a magnitude/EQ mismatch but phase/excitation character, which motivated the BigVGAN vocoder.

an exact match for our pipeline (24 kHz, hop 256, $f_{\text{max}} = 12$ kHz).

Because our magnitude spectrum already matches the reference, we integrate BigVGAN by *re-vocoding*: compute a mel from the acoustic model’s output and let BigVGAN regenerate the waveform. The mel preserves our (correct) magnitude while discarding HiFi-GAN’s flawed phase; BigVGAN synthesizes clean phase and excitation. This requires *no* fine-tuning of BigVGAN. Informal and objective evaluation (§10) confirm the breathiness is removed and texture reaches the reference level, at a real-time factor still far below one (the added vocoder RTF is ≈ 0.07). We release both the native HiFi-GAN model (for minimal-dependency, single-model deployment) and the BigVGAN pipeline (for maximum fidelity).

9 Experimental Setup

Data. We train on `lahgtna-levantine-tts` (CC-BY-4.0): 50,000 clips, 66.8 h, 24 kHz, 10 speakers, drawn from the Shami Levantine corpus plus $\sim 12\%$ synthetic code-switch sentences. Text is partially diacritized. For the single-speaker system evaluated here we use one speaker (“Badr,” ~ 6.2 h after filtering to $[0.8, 12]$ s clips), phonemized offline through the v2 front-end. A 40-utterance held-out set spans pure Levantine and code-

switching.

Model & training. ShamiVITS has $\sim 36\text{M}$ parameters; the HiFi-GAN discriminator 47M; BigVGAN 112M. Training uses bf16 autocast, AdamW ($\beta = 0.8, 0.99$), batch 12–16, `seg_size` 8192–12288, on a single RTX 3060 (12 GB); loss weights $c_{\text{mel}} = 45$, $c_{\text{kl}} = 1$, $c_{\text{dur}} = 1\text{--}2$, $c_{\text{fm}} = 4$, $c_{\text{stft}} = 3$. Each stage warm-starts from the previous best; checkpoints (generator *and* discriminator) are written atomically to survive interruption.

Metrics. We report ASR round-trip CER/WER via Whisper large-v3 [8] (Arabic forced, text normalized by diacritic/alef/ya/ta-marbuta folding), duration ratio, high-frequency spectral flatness against the reference, and real-time factor (RTF). All three systems (baseline / ours-HiFi-GAN / ours-BigVGAN) are synthesized at each model’s duration-calibrated `length_scale` and scored identically.

10 Results

Headline. Table 1 shows Levantine Arabic CER falling from 0.751 to **0.106** (−86%) and WER from 0.993 to **0.347** (−65%), while overall CER drops to 0.31. Both our systems run in real time ($\text{RTF} \leq 0.09$, i.e. $\geq 11\times$ faster than real time). BigVGAN further improves pure-Levantine CER and WER over HiFi-GAN and, more importantly, removes the perceptual breathiness (below).

Ablation. Figure 5 traces CER across the contribution sequence. The error drops in two clear moves — matched Levantine data + labels (0.75 $\rightarrow$ 0.46) and the deterministic duration head (0.46 $\rightarrow$ 0.11) — after which it is *flat*: 24 kHz, feature-matching, and BigVGAN hold CER while improving *fidelity and texture*, which CER does not measure. This two-regime structure (intelligibility first, then perceptual quality) is the paper’s central empirical story.

Duration and texture. The deterministic head yields a duration ratio of 0.97 at natural pacing (Fig. 2a). High-frequency spectral flatness (Fig. 2b) shows both our decoders at the reference level — magnitude-domain quality is saturated. The audible HiFi-GAN $\rightarrow$ BigVGAN gap is in *phase*: Figure 4 confirms our average and noise-floor magnitude spectra already track the reference within a few dB, which is why we localized the residual defect to the vocoder rather than the acoustic model.

Robustness. On eight held-out *novel* sentences (up to 140 tokens, 13 language spans, numbers, dates, currency, technical English) the full pipeline produced valid, naturally-paced audio for all eight, indicating the system generalizes beyond the evaluation sentences rather than overfitting them.

Real-time factor. Figure 6 decomposes RTF. The acoustic model is ~ 0.02 ; BigVGAN adds ~ 0.07 ; the total ~ 0.085 leaves ample headroom for streaming on commodity hardware.

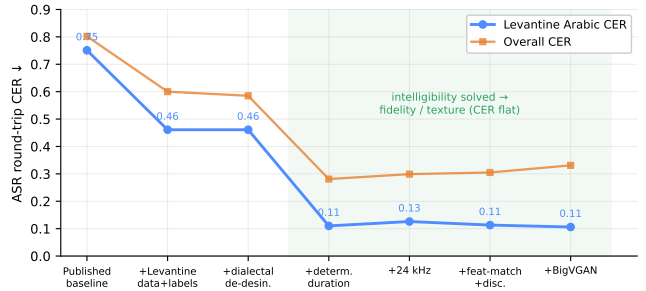


Figure 5: CER across the v2 contribution sequence (end-points are the consistently-measured systems of Table 1; intermediate points are development measurements). Intelligibility is solved by matched data and deterministic duration; later steps (shaded) trade nothing in CER while improving fidelity and texture.

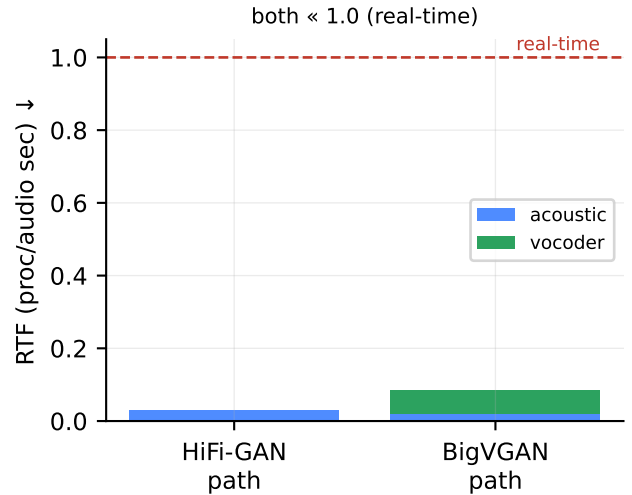


Figure 6: Real-time factor. The full BigVGAN pipeline (acoustic + vocoder) stays far below the real-time line on a single RTX 3060.

11 Discussion

Two findings generalize beyond Levantine. First, *duration modeling dominates perceived naturalness and, indirectly, measured intelligibility*: a well-trained acoustic model can sound robotic and score poorly purely because a mis-calibrated duration predictor forces a prosody-flattening global stretch. A deterministic head is a small, low-risk change with an outsized effect. Second, *magnitude metrics saturate before perceptual quality does*: our spectral-flatness and average-spectrum measures declared parity with the reference while listeners still heard vocoder breathiness. The defect was phase, and only an anti-aliased vocoder (BigVGAN) closed it. Practitioners chasing the last increment of naturalness should treat magnitude-domain losses/metrics as necessary but not sufficient and budget for a phase-aware vocoder.

We also stress the *data-labeling* lever specific to dialect-

Table 1: Main results on the held-out set (ASR round-trip via Whisper large-v3; lower is better). Ours is the single-speaker ShamiVITS with the deterministic duration head, 24 kHz, and the texture recipe; “+BigVGAN” re-vocodes its output. Best per column in **bold**.

System	Pure Levantine		Code-switch		Overall		System	
	CER	WER	CER	WER	CER	WER	SR	RTF
Published baseline (16 kHz, stochastic dur.)	0.751	0.993	0.853	0.993	0.802	0.993	16k	0.024
Ours, HiFi-GAN decoder (24 kHz)	0.113	0.377	0.498	0.713	0.305	0.545	24k	0.028
Ours, + BigVGAN vocoder	0.106	0.347	0.556	0.806	0.331	0.576	24k	0.085

tal Arabic. The gap between “the front-end can produce Levantine phonemes” and “the front-end *does*, on real bare text” was 0.4% vs. 100% of clips; closing it (automatic diacritization plus de-desinentialization) was as important as any acoustic change.

12 Limitations

Our evaluation set is small (40 clips), so absolute CER/WER carry noise of order ± 0.02 ; we therefore emphasize *relative* differences and released audio over precise absolute values. ASR round-trip CER via Whisper has a floor well above zero on dialectal Arabic — Whisper is MSA-biased and penalizes correct Levantine pronunciations — so it under-states true quality and mis-scores the English portions of code-switch (visible as the one column where BigVGAN’s CER rises); human MOS would be a better instrument and is future work. The BigVGAN path currently re-vocodes (running two vocoders); a cleaner design predicts the mel directly, which we leave to v3 along with multi-speaker conditioning (the data has 10 speakers; the single-speaker backbone lacks speaker-conditioning layers). Finally, the base `mms-tts-ara` is released under a non-commercial license, which the derived weights inherit.

13 Conclusion

Shami-TTS turns a promising but robotic Levantine/English code-switching TTS baseline into a production-grade, real-time system on a single consumer GPU. Four targeted changes — a dialectal front-end with automatic diacritization and de-desinentialization, a deterministic duration head, a 24 kHz decoder with an accumulated-adversarial texture recipe, and an anti-aliased BigVGAN vocoder — cut Levantine CER by 86% and WER by 65% while reaching reference-level texture and $\geq 11\times$ real-time speed. We release code, both decoder checkpoints, and audio. The remaining frontier — multi-speaker breadth and a single-pass mel→BigVGAN model — is well scoped for v3.

Reproducibility

Code, configs, checkpoints (HiFi-GAN and BigVGAN pipeline), the 40-clip eval set, and all audio are released. The pipeline is: `prepare_lahgtna.py` (extract + phonemize) → `finetune_gan.py` `--deterministic-duration` `--sample-rate 24000` `--c-fm 4` `--c-stft 3` → `eval_checkpoint.py` `--auto-ls` `--asr` → `bigvgan_vocoder.synthesize`. All experiments use a single RTX 3060 (12 GB).

A Levantine G2P: dialect rules

Table 2 lists the headline Levantine realizations encoded in the rule-based G2P. These are the features most audible to a native listener and the ones an MSA G2P gets wrong. The rule engine additionally applies emphatic vowel backing ($\text{a}, \text{a} \rightarrow \text{a}, \text{a}:$ near ق ط ظ ص ض ص), CCC epenthesis, definite-article sun/moon assimilation, and a high-frequency function-word lexicon.

Table 2: Selected Levantine G2P rules (grapheme / context $\rightarrow$ IPA), with a spoken example. MSA realizations are shown for contrast.

Grapheme/context	Levantine	(MSA)	Example
ق <i>qaf</i>	ʔ	q	قلب $\rightarrow$ ʔalb “heart”
ج <i>jim</i>	ʒ	dʒ	جبنه $\rightarrow$ ʒibne “cheese”
ث <i>tha</i>	t	θ	(urban stop merger)
ذ <i>dhal</i>	d	ð	(urban stop merger)
ظ <i>dha</i>	zʕ	ðʕ	emphatic
ة final <i>ta-marb.</i>	e	a	imala; a after gutturals
ay, aw	eː, oː	aj, aw	بيت $\rightarrow$ beːt, يوم $\rightarrow$ joːm
case ending (icrab)	∅	-u, -i, -a	de-desinentialization
شو	ʃuː	—	lexicon
بدي	biddi	—	lexicon “I want”
هلق	hallaʔ	—	lexicon “now”

B Front-end worked examples

Table 3 shows the front-end output for representative inputs, including a glued code-switch and a numeral. The language-ID stream (A=Arabic, E=English, .=neutral/boundary) is emitted in lockstep with the phonemes.

Table 3: Text $\rightarrow$ IPA (+ language-ID sketch) from the deterministic front-end.

Input	IPA	lang stream
بكرّا London بيروت من flight إيجز بدي	ˈbiddi ʔiħziz flˈaɪt min beːruːt tə lˈʌndən bukra	A A E A A E E A
لشّ training مشّ ال on the server؟	leːʃ ʔil tɪˈeɪnɪŋ mɪʃ ʃaɣɣaːl ɔnðə sˈɜːvə	A A E A A E E
عندي meeting الساعة 3:30	ʃindi mˈiːtɪŋ ʔissaːʕa θalaːθe wnuːsˈʕ	A E A A(num)

C Hyperparameters

Table 4: Training configuration (single RTX 3060, 12 GB, bf16).

optimizer	AdamW (0.8, 0.99)	learning rate	2×10^{-4}
batch size	12–16	seg_size	8192–12288
c_{mel}	45	c_{kl}	1
c_{dur}	1–2	c_{fm}	4
c_{stft}	3	MR-STFT ffts	512/1024/2048
mel bins	80 (loss)	sample rate	24,000 Hz
hop / n_{fft}	256 / 1024	grad clip	10
steps/stage	8k–10k	save every	1k (atomic)
VITS params	~36 M	disc. / BigVGAN	47 M / 112 M

D Data composition and per-stage checkpoints

The corpus (lahgtna-levantine-tts, CC-BY-4.0) is 50,000 clips / 66.8 h / 24 kHz, 10 speakers (5M/5F), ~88% pure Levantine and ~12% synthetic code-switch, partially diacritized. Single-speaker experiments use speaker *Badr* (5,000

clips, 6.2 h after $[0.8, 12]$ s filtering; 40-clip balanced held-out set). Each contribution is a warm-started fine-tune of the previous best; checkpoints (generator + discriminator) are released for baseline, deterministic-duration (16 kHz), 24 kHz, texture, and the BigVGAN pipeline.

References

- [1] J. Kim, J. Kong, J. Son. “Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech.” *ICML*, 2021.
- [2] J. Kong, J. Kim, J. Bae. “HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis.” *NeurIPS*, 2020.
- [3] S. Lee, W. Ping, B. Ginsburg, B. Catanzaro, S. Yoon. “BigVGAN: A Universal Neural Vocoder with Large-Scale Training.” *ICLR*, 2023.
- [4] L. Ziyin, T. Hartwig, M. Ueda. “Neural Networks Fail to Learn Periodic Functions and How to Fix It.” *NeurIPS*, 2020.
- [5] R. Yamamoto, E. Song, J.-M. Kim. “Parallel WaveGAN: A Fast Waveform Generation Model based on Generative Adversarial Networks with Multi-Resolution Spectrogram.” *ICASSP*, 2020.
- [6] J. Kim, S. Kim, J. Kong, S. Yoon. “Glow-TTS: A Generative Flow for Text-to-Speech via Monotonic Alignment Search.” *NeurIPS*, 2020.
- [7] V. Pratap et al. “Scaling Speech Technology to 1,000+ Languages.” *arXiv:2305.13516*, 2023.
- [8] A. Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision.” *ICML*, 2023.
- [9] O. Obeid et al. “CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing.” *LREC*, 2020.
- [10] M. Saeed et al. “CATT: Character-based Arabic Tashkeel Transformer.” *ArabicNLP*, 2024.
- [11] eSpeak NG contributors. “eSpeak NG Text-to-Speech.” <https://github.com/espeak-ng/espeak-ng>, 2016–.